

Model Comparison: Foundation Models vs. Gradient-Boosted Trees

This document is the dedicated results comparison for the eight notebooks in this repository. It does not repeat the theory behind each model — see [HANDBOOK.md](#) for that. Every number below was read directly from an executed notebook’s output; none are estimated, vendor-cited, or carried over from an external benchmark.

A critical honesty note before anything else: this repository contains *two* separate, internally-comparable groups, plus a first, standalone pair of notebooks. They are **not** all comparable to each other:

1. **Google TabFM’s own pair** — Adult Income (classification) and California Housing (regression). Only compared against its own in-notebook baselines.
2. **The regression trio** — TabICLv2, CatBoost, LightGBM — all on the *same* NYC Green Taxi tip dataset, same 80/20 split, same 3,000-row held-out evaluation set (`random_state=42` throughout). Directly comparable to each other.
3. **The classification trio** — TabPFN v3, Mitra, XGBoost — all on the *same* Forest Covertype dataset, same split, same 3,000-row evaluation set. Directly comparable to each other.

Comparing group 1’s numbers against group 2 or 3 (e.g. TabFM’s California Housing RMSE against LightGBM’s NYC Taxi RMSE) would be meaningless — different datasets, different targets, different scales. We don’t do that anywhere in this document.

Regression trio: NYC Green Taxi tip prediction

Dataset: 581,835 real trip records (December 2016), cleaned to 578,293 after dropping invalid negative tips and implausible trip durations. Train/test split 80/20 (`random_state=42`), evaluated on the same 3,000-row held-out sample throughout. Target: `tip_amount` in dollars — zero-inflated (~15% exact zeros), right-skewed, with a genuine leakage trap (`total_amount`) deliberately excluded from every model’s features.

Model	RMSE ↓	MAE ↓	R ² ↑	Wall-clock	Context / training size
LinearRegression (baseline, all 3 notebooks)	2.6538	1.0855	0.2120	5.6–5.9s	Full pool (462,634 rows)
TabICLv2	2.5694	1.0756	0.2613	7.6s	20,000-row context
CatBoost	2.3301	1.0293	0.3925	26.2s	Full pool (462,634 rows)
LightGBM	2.2608	1.0345	0.4281	1.9s	Full pool (462,634 rows)

What actually happened here

- **LightGBM won on both accuracy and speed.** Best RMSE/R² *and* the fastest wall-clock time in the entire trio, by a wide margin (1.9s vs. CatBoost’s 26.2s, both training on the identical 462,634-row pool). This is a genuinely clean result, not a close call.
- **CatBoost was a close second on accuracy**, at roughly 14x LightGBM’s training time. Its ordered-boosting categorical handling clearly worked — both GBMs comfortably beat the linear baseline on the two high-cardinality location columns without any manual encoding.
- **TabICLv2, with only a 20,000-row context (4.3% of the full training pool), still beat the full-data linear baseline** and did so in 7.6 seconds — a real demonstration of what “zero-shot, no

tuning, small context” can buy you, even though it didn’t catch the full-data GBMs on this particular dataset.

- **Nothing here is cherry-picked.** All four rows are read from the same evaluation set; no model was given an advantage the others didn’t have access to (the GBMs’ “advantage” — the full training pool — is real and disclosed, not hidden).

Why the foundation model didn’t win: this dataset’s predictive signal leans heavily on the two very-high-cardinality location columns (233 and 259 levels) interacting with trip duration and time-of-day — exactly the kind of fine-grained, per-category structure that a GBM trained on 400,000+ rows can memorize category-by-category, while a 20,000-row in-context sample simply hasn’t seen enough examples of every zone pair to do the same. This is a real, structural limitation, not an implementation gap — see [HANDBOOK.md](#) for the deeper discussion of context-size tradeoffs.

Classification trio: Forest Covertypes (7-class)

Dataset: 581,012 real cartographic records, 7 forest cover types, meaningfully imbalanced (211,840 rows of the majority class vs. 2,747 of the rarest — a 77x ratio). Wilderness area and soil type were reconstructed from scikit-learn’s one-hot encoding into two compact categorical columns (4 and 40 levels respectively). Same 80/20 split, same 3,000-row stratified evaluation set throughout.

Model	Accuracy ↑	Balanced accuracy ↑	F1 (macro) ↑	Log loss ↓	Wall-clock	Context / training size
LogisticRegression (baseline, all 3 notebooks)	0.7317	0.5237	0.5428	0.6285	29.2s	Full pool (464,809 rows)
Mitra (zero-shot)	0.7347	0.3959	0.3876	0.6398	9.4s	4,000-row context
Mitra (fine-tuned, 30 steps)	0.8000	0.6094	0.6500	0.4834	108.5s (11.5x zero-shot)	4,000-row context
XGBoost (default-ish)	0.9190	0.9248	0.9263	0.2171	247.3s	Full pool (464,809 rows)
XGBoost (tuned)	0.9767	0.9641	0.9605	0.0667	71.2s + 55s tuning	Full pool (464,809 rows)
TabPFN v3	<i>not executed — see note below</i>					

What actually happened here

- **A 55-second RandomizedSearchCV pass (8 configs, 3-fold, on a 30,000-row subsample) took XGBoost from 91.9% to 97.7% accuracy** — the single clearest “tuning matters” result in this entire repository. The untuned default configuration alone already thoroughly beat the linear baseline; a genuinely light tuning effort closed most of the remaining gap to near-ceiling performance on this benchmark.
- **Mitra was deliberately tested within its own documented operating envelope** — a 4,000-row context, matching its publisher’s stated sweet spot of “under 5,000 rows, under 100 features” — rather than the full 580,000-row pool. Zero-shot, it barely edged out the linear baseline on raw accuracy (0.7347 vs. 0.7317) but was noticeably *worse* on balanced accuracy (0.396 vs. 0.524) — it struggled disproportionately on the rarer cover types despite matching on overall accuracy, a real finding you’d miss if you only looked at accuracy.

- **Fine-tuning Mitra for just 30 steps (108.5s, 11.5x the zero-shot wall-clock) closed most of that gap and then some** — accuracy rose to 0.800 and balanced accuracy to 0.609, clearing the linear baseline on both metrics. This is the one head-to-head in this repository that directly shows fine-tuning’s value on a foundation model, since Mitra is the only model here that supports it. It still falls well short of XGBoost trained on the full 464,809-row pool — an honest, expected result given Mitra was intentionally kept inside its small-data envelope rather than given the full dataset.
- **TabPFN v3’s notebook is fully written but not executed in this repository.** As of this writing, the `tabPFN` package requires a one-time interactive license acceptance (see the README’s “[TabPFN v3: one-time license step](#)”) that cannot be automated. Anyone with a Prior Labs account and API token can run it themselves — the code, theory, and analysis are complete and ready to go.

Why tuning had such a large effect here: Covertypes’s decision boundary between cover types is genuinely non-linear and depends on interactions between elevation, aspect, and soil type that a shallow, under-regularized tree ensemble captures poorly. `max_depth` and `n_estimators` in particular moved the needle a lot in our search — a concrete, reproducible illustration of why “just run XGBoost with defaults” and “run a properly tuned XGBoost” are two different baselines, and papers/benchmarks that only report one of them should be read carefully.

Google TabFM’s own results (not part of either trio)

Adult Income (binary classification, 48,842 rows):

Model	Accuracy	ROC-AUC	Training data
LogisticRegression	0.866	0.9154	Full pool (39,073 rows)
XGBoost	0.880	0.9355	Full pool (39,073 rows)
TabFM (fast preset)	0.878	0.9247	2,000-row context
TabFM (ensemble preset)	0.878	0.9231	2,000-row context

California Housing (regression, 20,640 rows):

Model	RMSE	R ²	Training data
LinearRegression	0.7024	0.6108	Full pool (16,512 rows)
XGBoost	0.4733	0.8232	Full pool (16,512 rows)
TabFM (fast preset)	0.4287	0.8550	2,500-row context
TabFM (ensemble preset)	0.4298	0.8542	2,500-row context

TabFM’s own ensemble preset (7x the wall-clock of its fast preset, per the `google_tabfm_*` notebooks) did not improve on the fast preset in either task — the same pattern the regression and classification trios also show: **heavier inference-time ensembling is not a free accuracy win**, and should always be measured against its own fast-preset baseline before assuming it’s worth the extra cost.

Cross-cutting takeaways

1. **A tuned classical GBM, given the full dataset, is still the strongest single result in this repository** — 97.7% accuracy on Covertypes, best RMSE and by far the fastest training on the NYC Taxi regression task. Nothing here overturns that.
2. **Foundation models are competitive on a first pass, with zero tuning, using a fraction of the data.** TabFM beat full-data XGBoost outright on California Housing. TabICLv2 beat the full-data linear baseline using under 5% of the training pool, in single-digit seconds.

3. **“Ensemble”/heavier presets on foundation models did not pay for themselves** in either TabFM comparison we ran — a genuinely useful, reproducible finding for anyone deciding whether to spend the extra compute.
4. **Context-size and licensing constraints are real, model-specific, and worth checking before you start** — TabPFN v3 needed a manual license step we couldn’t automate; Mitra’s own documentation steered us toward a small context; TabICLv2 and Google TabFM each have their own hard limits (documented in [HANDBOOK.md](#)).
5. **Tuning effort is not optional if you want a fair comparison.** The single largest accuracy jump in this entire repository (91.9% → 97.7%) came from tuning a classical model, not from switching model families.
6. **Fine-tuning a foundation model can matter as much as tuning a GBM.** Mitra’s 30-step fine-tuning run (108.5s) took it from barely beating the linear baseline on accuracy — while actually being *worse* on balanced accuracy — to clearing the baseline on both. Zero-shot alone would have understated what the same pretrained weights are capable of.
7. **Accuracy alone can hide a real problem.** Zero-shot Mitra’s accuracy (0.7347) looked competitive with the linear baseline (0.7317) until balanced accuracy (0.396 vs. 0.524) revealed it was doing meaningfully worse on the rarer classes — always check a balanced/macro metric on an imbalanced target, not just raw accuracy.

A PDF export of this document is available at [COMPARISON.pdf](#).